

DeFiFlowBench: Evaluating Safe Executability in Natural-Language DeFi Workflow Synthesis

Anonymous Authors
Paper under review

Abstract—Natural-language interfaces for decentralized finance (DeFi) promise to make multi-step protocol interactions accessible to non-expert users, but they are typically evaluated only on whether the generated workflow graph looks structurally valid. We argue that this is the wrong bar for a high-stakes execution domain, and we study a stricter question: are generated DeFi workflows *safely executable*? We introduce DEFIFLOWBENCH, a benchmark of 120 expert-authored prompts spanning swaps, limit orders, cross-chain transfers, and compositional tasks, with difficulty tiers, paraphrase clusters, and adversarial cases. Each generated workflow is scored on three nested static layers—structural validity, executable configuration completeness, and declared safety—and on an execution layer that runs each generated swap on a real EVM. To make scores interpretable we bound the scale with an oracle solver (which scores 1.00 on every layer and is on-chain safe) and null and random-node baselines (which score 0.00), and we validate the static safety proxy against on-chain outcomes: on our data the proxy never certifies a workflow that then executes unsafely. Evaluating a deterministic template, a deployed no-code DeFi generator, and direct, constrained, few-shot, and safety-instructed language-model baselines across two model families, we find a large and consistent structural-to-safe gap: the best system reaches only 0.44 structural validity and no system exceeds 0.29 safe-executability (95% CI [0.21, 0.38]). The failure modes are complementary and quantified: the template binds no trade parameters, the deployed generator breaks required structure, and language models parse parameters but omit safety machinery. On-chain, direct, constrained, and few-shot configurations each execute 10–13 swaps at roughly 33% self-inflicted price impact because they wire a slippage bound but no price-impact gate. An explicit safety instruction is the only intervention that eliminates on-chain unsafe execution—yet still leaves most workflows structurally incomplete—showing the gap is real but partially addressable. We release the benchmark, baselines, execution harness, and analysis code.

Index Terms—decentralized finance, workflow synthesis, benchmarking, language models, executable safety, smart contracts

I. INTRODUCTION

Natural-language workflow generation is attractive for decentralized finance (DeFi) because common user goals—token swaps, limit orders, and cross-chain transfers—require coordinating multiple protocols and many configuration choices. A system that turns “swap 1 ETH to USDC with at most 1% slippage” into an executable workflow could make DeFi accessible to users who cannot write contract calls by hand.

Evaluations of such systems, however, typically stop at whether the generated workflow, viewed as a directed graph of nodes and edges, is structurally valid. We argue that this is the wrong bar for DeFi. A graph that looks correct may

still be unsafe or non-executable if it omits the concrete trade parameters needed to run, ignores price-impact limits, uses unsupported chains or tokens, or fails to gate execution against the cost of the user’s own order. In a domain where a single transaction can move real funds irreversibly, structural validity is necessary but far from sufficient.

This paper reframes DeFi workflow synthesis as a benchmark and measurement problem centered on *safe executability*. We evaluate generated workflows on three nested static layers—structural validity, executable configuration completeness, and declared safety—and, crucially, add an execution layer that runs each generated swap on a real EVM and observes the on-chain outcome. This lets us separate three questions that prior evaluations conflate: does the graph have the right shape, does it carry the parameters needed to execute, and, when executed, does it actually behave safely?

Our central finding is that structural validity does not imply safe executability, and the gap is large and consistent. Across a 120-prompt benchmark and a range of systems—a deterministic template baseline, a deployed no-code DeFi generator, and direct, constrained, few-shot, and safety-instructed language-model baselines on two model families—the best system reaches only 0.44 structural validity and none exceeds 0.29 safe-executability. On-chain, the direct, constrained, and few-shot configurations each execute 10–13 large swaps at roughly 33% self-inflicted price impact because they set a slippage bound but no price-impact gate. An explicit safety instruction is the only intervention that removes on-chain unsafe execution, and even it leaves most workflows structurally incomplete.

a) Contributions.: (1) We introduce DEFIFLOWBENCH, a 120-prompt benchmark over four DeFi task categories with difficulty tiers, paraphrase clusters, and adversarial cases, and a three-layer static evaluation (structural, executable, safe) plus a clarification axis, with safety predicates derived uniformly across systems. (2) We add an on-chain execution layer that runs generated swaps on a real EVM and distinguishes a slippage bound from a price-impact gate, turning “declared safety” into “observed safety”, and we show the static safety proxy is sound against this on-chain ground truth (no false certifications). (3) We calibrate the scale with an oracle ceiling and null/random floors, report every rate with confidence intervals, and provide a quantified failure taxonomy and robustness analysis; we ablate constrained, few-shot, and safety-instructed prompting to show the gap is real but only partially closable by prompt design. (4) We release the prompts, gold

annotations, baselines, execution harness, and analysis code for reproducible evaluation.

II. RELATED WORK

a) Workflow and agentic generation benchmarks.: Recent work evaluates whether language models can generate multi-step workflows. WorFBench [1] benchmarks agentic workflow generation and scores generated workflows largely at the level of graph structure and node/edge correctness. Our work is complementary but distinct: we argue that for a high-stakes execution domain such as DeFi, graph-level correctness is necessary but insufficient, and we add executable and on-chain safety layers on top of structural scoring.

b) Tool-use and API evaluation.: A related line evaluates LLMs as tool users. ToolLLM [2] and API-Bank [3] measure whether models can select and invoke external APIs to satisfy instructions. These benchmarks emphasize correct API selection and invocation; they do not model domain-specific safety predicates such as price-impact gating, nor do they execute the resulting plans against a stateful financial system. We adopt the tool-use framing but specialize the evaluation to safety-constrained on-chain execution.

c) Decentralized finance and automated market makers.: Our execution layer builds on the constant-product automated market maker design introduced by Uniswap [4], whose $x \cdot y = k$ invariant makes price impact an explicit, computable function of trade size. This is precisely the property that lets us distinguish a slippage bound from a price-impact gate and observe unsafe execution directly.

d) No-code DeFi workflow systems.: We evaluate an existing no-code DeFi platform (Koan) as a first-class baseline rather than a strawman, using its real intent parser and workflow generator. To our knowledge, prior no-code DeFi tooling has not been evaluated against an executable safety benchmark, which is the gap this paper addresses.

III. TASK AND EVALUATION MODEL

We formalize DeFi workflow synthesis as mapping a natural-language intent p to a workflow specification $W = (N, E, C)$, where N is a set of typed nodes drawn from a fixed DeFi node vocabulary (e.g. `tokenSelector`, `oneInchQuote`, `priceImpactCalculator`, `oneInchSwap`, `limitOrder`, `fusionPlus`, `transactionMonitor`), $E \subseteq N \times N$ is a set of type-level data-flow edges, and C is an execution configuration mapping keys such as `fromToken`, `amount`, `slippage`, and `targetPrice` to values.

Each benchmark item pairs a prompt with an expert-authored gold annotation (N^*, E^*, C^*, S^*) , where S^* is a set of *safety predicates* the workflow must satisfy to be safely executable (e.g. a slippage bound, a price-impact gate, transaction monitoring, a distinct bridge destination). The annotation also marks whether a prompt is underspecified, in which case the ideal system response is to request clarification rather than emit a workflow.

A. Three static evaluation layers

We score a generated workflow along three strictly nested layers, so that each captures a distinct failure mode.

a) Structural validity.: A workflow is *graph-valid* if it recalls every required node type and every required type-level edge and the generator did not error. This layer is deliberately lenient: it ignores extra nodes unless they break a required edge, matching the weak notion of “valid” that graph-only evaluations reward.

b) Executable completeness.: A workflow is *executable* if it is graph-valid *and* every required configuration key in C^* is populated with a concrete (non-placeholder) value. A node that merely exists in template mode, with no trade amount or target price, is not executable.

c) Declared safety.: A workflow is *statically safe* if it is executable *and* every required safety predicate in S^* is declared and parameterized. Safety predicates are derived uniformly from the normalized workflow for all systems, so no system is advantaged by self-reporting safety.

A separate *clarification* axis scores underspecified prompts: the correct behavior is to ask for the missing information, not to emit a confidently wrong workflow.

B. On-chain execution layer

Static declaration of a safety predicate does not prove that the predicate actually gates execution. We therefore add an execution layer that runs each generated swap on a real EVM. We deploy a constant-product automated market maker (Uniswap-V2-style $x \cdot y = k$ with a 0.3% fee) and mock ERC20 tokens on an in-process EVM, seed deterministic reserves, and submit the generated trade as an actual transaction. The harness observes the mined receipt, gas, or revert, and labels the outcome as one of `executed_safe`, `reverted_slippage`, `aborted_price_impact`, `unsafe_executed`, `pending_not_filled`, or `not_executable`.

The critical distinction the harness captures is between a slippage bound and a price-impact gate. A slippage bound sets a minimum output computed from the (already impact-adjusted) quote, so it protects only against price *movement between quote and execution*; it does *not* prevent a large order from executing at severe self-inflicted price impact. Only a separate price-impact gate does. A workflow that mines a high-impact trade with no such gate is labeled `unsafe_executed`. This is a deterministic local EVM snapshot, not a mainnet fork: it provides real EVM execution semantics (real `transferFrom`, real `require` reverts, real gas) but synthetic prices and liquidity (Section VIII).

IV. BENCHMARK

A. Categories and prompts

DEFIFLOWBENCH contains 120 expert-authored prompts across four categories: token swaps (40), limit orders (30), cross-chain transfers (30), and compositional workflows that combine primitives (20). Each prompt carries a difficulty tier (38 easy, 62 medium, 20 hard) and phenomena tags naming

the challenge it exercises. Fifteen prompts are underspecified (e.g. “help me trade some tokens”) or contradictory (a bridge or swap whose source and destination are identical) and are scored on a separate clarification axis; several are adversarial in that the user waives a safety constraint the gold annotation still requires (e.g. “I don’t care about slippage”). To measure robustness to surface form independently of task difficulty, the benchmark includes *paraphrase clusters*: multiple prompts that encode the same underlying task with different wording (spelled-out numbers, terse notation, verbose narration, injected typos). A 30-prompt development *pilot* split is retained for reference. Prompts vary from fully specified (“swap 1 ETH to USDC with at most 1% slippage”) to schematic (“make a token swap app for UNI to ETH”).

B. Gold annotations

Each prompt is paired with a gold annotation recording (i) the required node types, (ii) the required type-level edges, (iii) the required execution-configuration keys, (iv) the safety predicates that must hold, and (v) a set of *allowed extra nodes* that a richer-but-correct workflow may include without penalty. The annotation scheme, including what qualifies as a required node, edge, configuration key, and safety predicate, is documented in the artifact’s annotation guide so that the final split can be extended consistently. The node vocabulary is taken from an existing executable DeFi node catalog, ensuring every required node corresponds to a real executor rather than an abstract label.

C. Baselines

We evaluate three reference baselines that calibrate the scale and five system families, all scored by the identical evaluator:

- **Oracle (ceiling).** Reconstructs a correct workflow from the gold annotation—required nodes and edges, concrete values for every required configuration key, and the parameters needed for `derive_safety` to declare each required predicate. It should, and does, score 1.00 on every layer, proving the tasks are solvable and the scorer rewards a correct solution.
- **Null and Random (floor).** The null baseline emits an empty workflow; the random baseline emits a random-size sample of real nodes in a random chain with no configuration. Both establish the floor and guard against a gameable metric.
- **Template-only.** Emits the canonical workflow for the prompt’s category with static safety defaults, but performs no language understanding and cannot populate trade-specific configuration.
- **Koan.** The regex-fallback intent parser and workflow generator of an existing no-code DeFi platform, run offline and deterministically—a real deployed system, not a strawman.
- **Direct / Constrained / Few-shot / Safety-instructed LLM.** A language model prompted to emit a workflow graph and configuration as JSON. The variants differ only in the prompt: no constraints; injected per-category

structural and safety constraints; a single worked in-context example; and an explicit instruction to make the workflow safe to execute. Comparing the variants on a fixed model isolates the effect of each prompt-design intervention.

The LLM baselines are evaluated on two model families to test whether findings are model-specific. Model outputs are normalized (node and edge vocabularies filtered; index-based and name-based edge encodings both accepted) before scoring, and each raw model response is retained so that metrics can be re-derived deterministically without additional queries. Every reported rate is accompanied by a 95% Wilson confidence interval, and mean-valued metrics by a bootstrap interval.

V. EXPERIMENTS

We evaluate over the 120-prompt MAIN split. Of the 120 prompts, 15 are underspecified or contradictory and are scored on the clarification axis, leaving $n = 105$ prompts that should yield a workflow. LLM baselines run at temperature 0 on two model families, Gemini 3.1 Flash Lite and GPT-5.4 mini, through a common OpenRouter endpoint. Because temperature-0 API calls still exhibit minor run-to-run variation, exact decimals may shift on re-run; all tables are generated directly from saved outputs, and every raw model response is stored so metrics can be re-derived without re-querying.

A. The scale is calibrated by reference baselines

Before comparing systems we fix the meaning of the scores (Table I). The *oracle*, which reconstructs a correct workflow from the gold annotation, scores 1.00 on all three static layers and is safe on every executed swap, confirming that the tasks are solvable and that the scorer rewards a genuinely correct solution. The *null* (empty output) and *random-node* baselines score 0.00 on every layer, so the metric cannot be gamed by degenerate or padded graphs. Real systems are therefore compared against a demonstrated ceiling of 1.00 and a floor of 0.00.

B. Structural validity does not imply safe executability

The central result is a large and consistent gap between structural validity and safe executability. The strongest structural system, constrained Gemini, reaches only 0.44 graph-valid, and **no system exceeds 0.29 safe-executability** (safety-instructed Gemini, 95% Wilson CI [0.21, 0.38]); most systems sit at or near the floor. The three layers fail for structurally different, quantified reasons (Section VI). The template-only baseline is perfectly graph-valid (1.00) yet has zero configuration completeness: well-formed graphs with no executable trade parameters. The Koan generator is graph-valid on only 0.12 of prompts: across the split it leaves 92 required edges and 64 required nodes unmatched, over-generating swaps into a fixed multi-node suite that breaks the required `oneInchSwap`→`transactionMonitor` edge, omitting `tokenSelector` on cross-chain prompts, and collapsing

TABLE I

STATIC STRUCTURAL/EXECUTABLE/SAFE METRICS VS. ON-CHAIN EXECUTION ON THE MAIN SPLIT ($n=105$ WORKFLOW PROMPTS). SAFE (CFG) IS REPORTED WITH A 95% WILSON INTERVAL. FORK-UNSAFE COUNTS WORKFLOWS THAT EXECUTED ON A LOCAL PY-EVM AT $>5\%$ OWN-TRADE PRICE IMPACT WITH NO PRICE-IMPACT GATE; FORK SAFE-RATE IS OVER PROMPTS WITH A DEFINITE ON-CHAIN OUTCOME. ORACLE AND THE NULL/RANDOM BASELINES BOUND THE CEILING AND FLOOR. LLM BASELINES RUN AT TEMPERATURE 0.

System	Graph valid	Exec. (cfg)	Safe (cfg) [95% CI]	Fork unsafe	Fork safe-rate
Oracle (ceiling)	1.00	1.00	1.00 [0.96,1.00]	0	1.00
Null (floor)	0.00	0.00	0.00 [0.00,0.04]	0	n/a
Random nodes	0.00	0.00	0.00 [0.00,0.04]	0	n/a
Template-only	1.00	0.00	0.00 [0.00,0.04]	0	n/a
Koan (regex+gen)	0.12	0.00	0.00 [0.00,0.04]	0	n/a
Direct LLM (Gemini 3.1 FL)	0.17	0.11	0.03 [0.01,0.08]	13	0.68
Direct LLM (GPT-5.4 mini)	0.02	0.00	0.00 [0.00,0.04]	12	0.57
Constrained LLM (Gemini 3.1 FL)	0.44	0.25	0.10 [0.06,0.18]	13	0.68
Constrained LLM (GPT-5.4 mini)	0.06	0.05	0.02 [0.01,0.07]	13	0.59
Few-shot LLM (Gemini 3.1 FL)	0.30	0.15	0.10 [0.06,0.18]	13	0.68
Few-shot LLM (GPT-5.4 mini)	0.26	0.10	0.10 [0.06,0.18]	10	0.72
Safety-instruct LLM (Gemini 3.1 FL)	0.35	0.29	0.29 [0.21,0.38]	0	1.00
Safety-instruct LLM (GPT-5.4 mini)	0.13	0.08	0.08 [0.04,0.14]	0	1.00

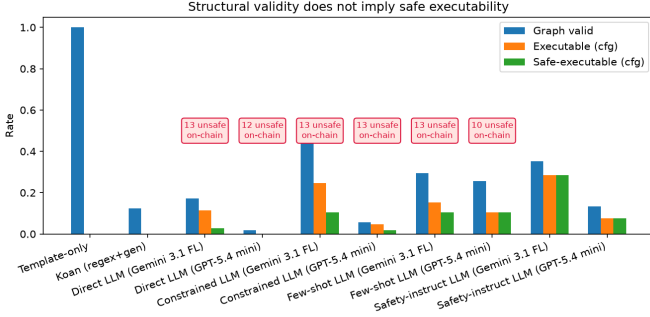


Fig. 1. Static structural, executable, and safe rates per system on the MAIN split ($n = 105$; reference floor/ceiling baselines omitted). Annotations mark workflows that executed unsafely on the local EVM.

compositional “swap and limit order” prompts to a limit-order-only workflow. The LLM baselines extract concrete configuration better but omit safety machinery, discussed next.

C. On-chain evidence of unsafe execution

The static safe rate treats a declared-but-unparameterized safety node as a failure, but it cannot show that an “executable” workflow is actually dangerous. The execution layer does. Under direct, constrained, and few-shot prompting, both models emit swaps with a concrete amount but no wired price-impact gate; on the local EVM these **mine successfully at roughly 33% self-inflicted price impact**, and the harness labels them `unsafe_executed`—10 to 13 such executions per configuration (Table I). This is the paper’s thesis made concrete: a slippage bound, which the models often set, computes its minimum output from the already impact-adjusted quote and so does not prevent a large order from executing at severe self-inflicted impact; only a separate price-impact gate does. The template and Koan baselines never reach this failure because they never populate a concrete amount, so nothing executes on-chain.

D. Does prompting close the gap? Ablations

We ablate two obvious mitigations on both models (Table I). Adding per-category structural *constraints* improves Gemini’s structural validity most ($0.17 \rightarrow 0.44$) but leaves 13 on-chain unsafe executions. A single in-context *few-shot* example helps GPT-5.4 mini’s structure sharply ($0.02 \rightarrow 0.26$) and modestly raises safe-executability, but still executes 10–13 unsafe swaps. Only an explicit *safety instruction* eliminates on-chain unsafe execution entirely for both models (fork safe-rate 1.00, zero `unsafe_executed`) by inducing a parameterized price-impact gate—yet it still leaves most workflows structurally incomplete (safe-executability 0.29 for Gemini, 0.08 for GPT). The gap is thus real but partially addressable by prompt design, and no single intervention closes both the structural and the safety half.

E. Clarification and per-category behavior

On the clarification axis, both LLM families correctly request clarification on most underspecified prompts (clarification-correct ≥ 0.73), whereas the template baseline never asks and Koan asks only when its keyword heuristics fall through (0.47). Per-category, structural validity is concentrated (Table II): Koan passes mainly the limit-order category, and the LLMs pass primarily swaps, with cross-chain and compositional prompts failing structural matching for nearly all systems.

VI. ANALYSIS

A. Construct validity: does the static proxy measure real safety?

The static safe-executable layer is a cheap proxy for an expensive property: whether a workflow is actually safe to run. We test the proxy against the on-chain outcome on every prompt the fork harness executed or gated to a definite safe/unsafe label ($n = 360$ system $\times$ prompt pairs across all runs). The proxy is *sound* on this data: it never labels

TABLE II
PER-CATEGORY GRAPH-VALID (G) AND SAFE-EXECUTABLE (S) RATE ON THE MAIN SPLIT. REFERENCE FLOOR/CEILING BASELINES (ALL CONSTANT) ARE OMITTED; SEE TABLE I.

System	Swap		Limit		Cross		Compo	
	G	S	G	S	G	S	G	S
Koan (regex+gen)	0.00	0.00	0.50	0.00	0.00	0.00	0.00	0.00
Direct LLM (Gemini 3.1 FL)	0.42	0.00	0.12	0.12	0.00	0.00	0.00	0.00
Direct LLM (GPT-5.4 mini)	0.03	0.00	0.00	0.00	0.04	0.00	0.00	0.00
Constrained LLM (Gemini 3.1 FL)	0.69	0.00	0.81	0.42	0.00	0.00	0.00	0.00
Constrained LLM (GPT-5.4 mini)	0.08	0.00	0.12	0.08	0.00	0.00	0.00	0.00
Few-shot LLM (Gemini 3.1 FL)	0.19	0.00	0.92	0.42	0.00	0.00	0.00	0.00
Few-shot LLM (GPT-5.4 mini)	0.19	0.08	0.69	0.31	0.08	0.00	0.00	0.00
Safety-instruct LLM (Gemini 3.1 FL)	0.72	0.56	0.38	0.35	0.00	0.00	0.06	0.06
Safety-instruct LLM (GPT-5.4 mini)	0.19	0.11	0.19	0.15	0.08	0.00	0.00	0.00

a workflow safe-executable that then executes unsafely on-chain (zero proxy-safe/on-chain-unsafe pairs). It is deliberately conservative—in 131 pairs it withholds “safe” from a workflow whose single executed swap happened to be within impact bounds—because it requires a declared price-impact gate rather than a fortunate trade size. This is the intended direction of error for a safety metric: it does not certify danger as safe. The 74 proxy-unsafe/on-chain-unsafe pairs are the concrete unsafe executions the proxy correctly refuses to certify.

B. Failure taxonomy

Aggregating the per-prompt scores into named failure categories (released in full with the artifact) separates the systems’ error profiles. The Koan generator’s failures are structural: 92 missing required edges and 64 missing required nodes, driven by fixed-suite over-generation and category collapse. The LLMs’ failures are predominantly *safety-predicate* omissions: for direct Gemini the most frequent missing predicate is `price_impact_gate` (54 prompts), followed by `transaction_monitoring` (45) and `slippage_bound` (34); GPT-5.4 mini additionally drops `transaction_monitoring` on 74 prompts by building terser graphs. The safety-instruction ablation nearly eliminates the two swap-safety omissions (`price_impact_gate` and `slippage_bound` fall to 1 each) while leaving cross-chain and limit-order predicates (`bridge_confirmation` 25, `price_bound` 16) largely untouched, explaining why it fixes on-chain swap safety but not the overall structural gap.

C. Robustness to surface form and difficulty

The benchmark holds task semantics fixed across paraphrase clusters and tags each prompt with a difficulty tier, letting us separate competence from surface sensitivity. Within paraphrase clusters, graph-valid agreement is high for the deterministic systems (Koan 0.91; safety-instructed Gemini 1.00) and lower for constrained Gemini (0.81), indicating that some of the LLM structural score is surface-sensitive rather than robust. Across difficulty tiers the expected monotone degradation appears for the stronger LLM configuration (constrained Gemini: easy 0.58, medium 0.35, hard 0.40), while

the rule-based generator is uniformly low (easy 0.16, medium 0.11, hard 0.00), consistent with keyword matching rather than compositional understanding.

VII. DISCUSSION

The benchmark exposes a consistent division of labor among failure modes. Deterministic template instantiation guarantees structure but cannot bind trade-specific parameters, so its outputs are inert. The deployed Koan generator, driven by pattern templates, over-generates and mis-routes, which breaks structural validity even where its intent classification is correct. Language models invert this trade-off: they parse the request well enough to fill in concrete amounts, tokens, and prices, but they do not reliably reproduce the required graph structure, and, most importantly, they do not wire the safety machinery that would make an executable trade safe.

The ablations refine rather than overturn this picture. Structural constraints and few-shot examples improve graph validity but leave the on-chain unsafe executions intact; only an explicit safety instruction induces the missing price-impact gate and drives on-chain unsafe execution to zero. Yet even that intervention leaves most workflows structurally incomplete, so no single prompt-level fix closes both halves of the gap. This asymmetry—safety parameterization is promptable, but robust structure is not—is itself a finding about where the difficulty lies.

The on-chain result sharpens the practical stakes. A workflow that a graph-only or even a configuration-level evaluator would accept as “executable” can, when actually run, mine a trade at severe self-inflicted price impact. This happens precisely because the systems conflate two different protections: a slippage bound, which guards against price movement between quote and execution, and a price-impact gate, which guards against the cost of one’s own order. Templates encode a static threshold but have no live trade to protect; language models supply the live trade but omit the gate. Neither composition is safe.

These observations suggest that safe DeFi workflow synthesis should be evaluated, and built, as a constraint-satisfaction problem over executable state rather than as graph generation. A generator that emits a structurally plausible, parameter-complete workflow is not done: the workflow must addition-

ally carry, and enforce, the safety predicates that the target protocols require. Our benchmark operationalizes exactly this requirement and makes the gap measurable.

VIII. LIMITATIONS

a) Local EVM, not a mainnet fork.: The execution layer deploys a constant-product AMM and mock tokens on an in-process EVM with deterministically seeded reserves. This yields real EVM execution semantics (real transfers, reverts, and gas) and byte-for-byte reproducibility, but the prices and liquidity are synthetic. It is a pinned local snapshot rather than a live mainnet fork; absolute price-impact magnitudes depend on the seeded reserves, though the qualitative unsafe-execution finding does not. Replacing the synthetic AMM with a mainnet-fork RPC is a natural extension and the harness interface is designed for that substitution.

b) Scale, single annotator, and scoring strictness.: The main split uses 120 expert-authored prompts, temperature-0 decoding, and a single run per model on two model families. Temperature-0 API calls still vary slightly across runs, so individual decimals are approximate; we report every rate with a confidence interval and generate all tables from saved outputs. The gold annotations were authored primarily by one team; we provide a second-annotator protocol, a reproducible stratified subset, and tooling to compute inter-annotator agreement, but the agreement number awaits an independent pass and is therefore not yet reported. Edge matching is strict and type-level, which penalizes reasonable-but-reordered graphs and treats the LLM structural rates as a lower bound; this does not affect the safe-rate or the on-chain unsafe-execution findings, which are driven by missing safety parameterization rather than edge ordering.

c) Category scope of the execution layer.: On-chain execution currently covers swaps and the swap leg of compositional workflows; limit orders are checked for immediate fillability against the pool mid-price, and cross-chain workflows are not executed because a single local chain cannot bridge. Every unexecuted case is recorded explicitly rather than silently passed.

IX. ETHICS AND RESPONSIBLE DISCLOSURE

This work is an offline evaluation. All experiments run against a local, in-process EVM with mock tokens and seeded liquidity; no transaction in this paper touches a live network or real funds. The benchmark is intended to help identify unsafe generated workflows before execution, not to encourage users to execute generated DeFi workflows with real assets. Any future extension to a mainnet-fork or live setting must keep simulation strictly separated from real execution, use test settings, and disclose residual risks. The evaluated no-code system and the language models are used as-is through their public interfaces; we report their failures to motivate safer synthesis, not to single out any implementation. Model outputs and derived metrics are released so that the results can be independently reproduced and audited.

REFERENCES

- [1] S. Qiao *et al.*, “Benchmarking agentic workflow generation,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.07869.
- [2] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world apis,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.
- [3] M. Li *et al.*, “API-Bank: A comprehensive benchmark for tool-augmented LLMs,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2304.08244.
- [4] H. Adams, N. Zinsmeister, and D. Robinson, “Uniswap v2 core,” Uniswap, Tech. Rep., 2020, whitepaper.